

Módulo 3: ASP.NET Core Razor Pages

Este módulo repasa la capa de presentación de la aplicación utilizando Razor Pages.

1. Arquitectura Page-Centric (**PageModel**)

En Razor Pages, cada vista `.cshtml` tiene una clase de respaldo `.cshtml.cs` derivada de `PageModel`.

- **OnGetAsync()**: Responde a peticiones HTTP GET (Carga la página).
- **OnPostAsync()**: Responde a peticiones HTTP POST (Envío de formularios).

2. ¿Por qué usamos Métodos Asíncronos (**async** / **await**)?

En aplicaciones web con ASP.NET Core, **todas las acciones que realizan operaciones de Entrada/Salida (I/O)** —como consultar SQLite o escribir en disco— deben ser **asíncronas** (**async Task**).

● Enfoque Sincrónico (Bloqueante - Mala Práctica en Web)

1. Llega una petición HTTP y ASP.NET Core le asigna un hilo del grupo de hilos (*Thread Pool*).
2. El hilo solicita datos a SQLite y **se queda congelado (bloqueado)** esperando a que el disco responda.
3. Mientras el hilo está congelado, no puede atender a otros usuarios. Si llegan 100 peticiones simultáneas, el servidor se queda sin hilos disponibles (**Agotamiento de Hilos / Thread Starvation**).

```
[Servidor Web] ---> Hilo #1 (BLOQUEADO esperando a la BD) ✕ --->
[SQLite]
```

● Enfoque Asíncrono (**async** / **await** - Buenas Prácticas)

1. Llega una petición HTTP y un hilo inicia la acción.
2. Al ejecutar **`await db.Tareas.ToListAsync()`**, **el hilo se libera inmediatamente** y vuelve al pool para atender a otros usuarios.
3. Cuando SQLite termina de leer el disco, un hilo libre retoma la ejecución del método donde se quedó.

```
[Servidor Web] ---> Hilo #1 libera el control al iniciar I/O --->
[SQLite]
Hilo #1 atiende a otros usuarios (MÁXIMA
ESCALABILIDAD) ✓
```

🎯 **Conclusión Didáctica:** El código asíncrono no hace que una sola petición individual sea mágicamente más rápida, sino que permite que el **servidor web atienda a muchísimos más usuarios en paralelo con los mismos recursos**.

2. El Patrón Post-Redirect-Get (PRG)

Para evitar el envío duplicado de formularios cuando el usuario presiona F5 o actualiza la página:

```
sequenceDiagram
    autonumber
    Usuario->>Servidor: POST (Envía formulario con datos)
    Servidor->>BaseDeDatos: Guarda la nueva Tarea
    Servidor-->>Usuario: HTTP 302 Redirect (Hacia /Index)
    Usuario->>Servidor: GET /Index (Carga lista actualizada)
    Servidor-->>Usuario: HTML Renderizado de la lista
```

Implementación en `Create.cshtml.cs`:

```
public async Task<IActionResult> OnPostAsync(CancellationToken ct)
{
    if (!ModelState.IsValid) return Page(); // Permanece en el formulario
    si hay errores

    await tareasService.CreateTareaAsync(new
    CreateTareaCommand(Input.Descripcion, Input.Fecha), ct);

    TempData["Mensaje"] = "Tarea creada correctamente";
    return RedirectToPage("./Index"); // Aplica PRG
}
```

3. Tag Helpers

Los **Tag Helpers** son atributos HTML especiales que se integran limpiamente con las clases de C#.

- **asp-for**: Vincula un elemento de entrada HTML con una propiedad del modelo C#, encargándose del binding y validaciones.
- **asp-page**: Genera la URL correcta para la navegación entre Razor Pages.
- **asp-validation-for**: Muestra mensajes de error de validación asociados al campo.

```
<form method="post">
    <label asp-for="Input.Descripcion">Descripción</label>
    <input asp-for="Input.Descripcion" class="form-control" />
    <span asp-validation-for="Input.Descripcion" class="text-danger">
</span>

    <button type="submit" class="btn btn-primary">Guardar</button>
</form>
```